

a.)

By first reading the count of numbers, I guaranteed that the memory allocation & iteration are controlled. This was used for the purpose of having the least amount of errors or miss-steps when writing my code. I came up with this code through the usage of the provided lecture slides in lab.

b.)

Unset

```
section .data
mode_read    db "r", 0 ; File mode for reading
fmt_input    db "%d", 0 ; Format for fscanf
fmt_output    db "Sum = %d", 10, 0 ; Format for printf
err_msg      db "Error opening file", 10, 0

section .bss
array        resd 1000 ; Space for 1000 integers
num_values    resd 1 ; Number of values in the file
sum          resd 1 ; Sum result
file_ptr      resd 1 ; File pointer storage

section .text
extern printf, fopen, fscanf, fclose
global main

main:
    push ebp
    mov ebp, esp

    ;; Get filename from command line (argv[1])
    mov eax, [ebp + 12] ; argv is at ebp+8, argv[1] at ebp+12
    mov eax, [eax + 4] ; First argument after program name

    ;; Open file
    push mode_read
    push eax ; Filename from command line
    call fopen
    add esp, 8

    test eax, eax ; Check if file opened successfully
    jz error_exit
    mov [file_ptr], eax ; Store file pointer

    ;; Read number of values
    push num_values
    push fmt_input
    push eax ; File pointer
```

```

    call fscanf
    add esp, 12

;; Read integers into array
mov ecx, [num_values] ; Number of integers to read
mov edi, array        ; Destination array
mov ebx, [file_ptr]   ; File pointer

read_loop:
    push ecx           ; Save counter
    push edi           ; Array position
    push fmt_input
    push ebx           ; File pointer
    call fscanf
    add esp, 12
    pop ecx            ; Restore counter

    cmp eax, 1         ; Check if read was successful
    jne close_file     ; Exit if fscanf fails

    add edi, 4         ; Move to next array position
    dec ecx            ; Decrease counter
    jnz read_loop; Continue if more numbers to read

;; Calculate sum
xor eax, eax ; Clear sum
mov ecx, [num_values] ; Number of integers
mov esi, array    ; Source array

sum_loop:
    add eax, [esi]    ; Add current number to sum
    add esi, 4        ; Move to next number
    loop sum_loop

    mov [sum], eax    ; Store final sum

;; Print result
push eax
push fmt_output
call printf
add esp, 8

close_file:
    ;; Close the file

```

```

    push dword [file_ptr]
    call fclose
    add esp, 4

    ;; Exit with success
    mov eax, 0
    mov esp, ebp
    pop ebp
    ret

error_exit:
    ;; Print error message
    push err_msg
    call printf
    add esp, 4

    ;; Exit with error
    mov eax, 1
    mov esp, ebp
    pop ebp
    ret

```

c.)

i.

```

[nthies1@linux6 proj2]$ ./assign2 randomInt100.txt
Sum = 4579

```

ii. **I've edited the provided text file for this additional output

```

[nthies1@linux6 proj2]$ ./assign2 randomInt100.txt
Sum = 3754

```